Intermediate Submission 3 Group 58

Sofia Constantinou, Dimitar Zhekov, Nicole Almeida, Stefanos Kritikos, Alex Christou, Efe Koç
Eindhoven University of Technology, The Netherlands
2127326, 2132966, 2087480, 2153785, 2075407, 2098156, Group 58

1 INTERMEDIATE SUBMISSION 3

1.1 Changes to Use Case and Requirements

The most important change we made in our main use case was that the main menu will also include a help screen option. This help screen is for the user to understand how the game is structured and played. The help screen will mostly be composed of a user manual, clearly describing how the game is played, while also explaining the rules of the game. explaining the features of the program itself. This change occurred because the game is complicated enough to follow, and thus the user might not be able to understand how it works without a user manual and a description of the game. Other than that, the use case diagram and descriptions made in the first submission have remained the same.

Regarding the requirements declared in the first phase of the project, there was a crucial change that occurred during the project implementation. After an extended discussion with the instructors, we concluded that the program should be able to run in JDK 21, rather than JDK 11, which is what we discussed in our constraint requirements. This happened due to the restrictions of the project stated by the course, which require the project to run in JDK 21. However, no other changes have been made so far. All functional and quality requirements have been followed so far, and so are the rest of the remaining constraints.

1.2 Project Structure

In the initial stages of the project, we decided to use the Model-View-Controller (MVC) framework to design our project in a clean and organized manner. It stood out from the other common frameworks due to our familiarity with it and its simplicity compared to the others. As some of us have a web developer background, it seemed rational not to reinvent the wheel and utilize our strengths. Furthermore, compared to the other most popular framework, Model-View-ViewModel (MVVM), our choice is straightforward and less complex to implement, which worked in our favor since some group members do not have extensive software engineering experience. We didn't just use MVC, though. We went even further in separating the concerns by also introducing services (a standard extension of MVC) and data transfer objects for the models. We chose this service-oriented approach to reduce complexity and coupling in the controllers, passing only the service interface and not the concrete implementation. The project also includes helper classes in the utils package and static resources, such as images, style sheets, fonts, sounds, and level data, in the resources package. We also have a test folder inside the source root, which matches the folder structure of the main application, ensuring that our tests are adequately defined and easily accessible. We have split the service package into several sub-packages to provide a more explicit separation of roles and clarity. We defined all of our core services as interfaces, and we have concrete implementations in the corresponding implementation (impl) folder. We did the view or the GUI with the help of the JavaFX library. The service and the controller are framework-agnostic, allowing them to work with any frontend library. The application starts from the Launcher class, which launches the JavaFX application defined in the App class (required because of Netbeans compatibility issues). By adapting our current project structure, we can easily achieve testable, decoupled, and maintainable code with a clear separation between application responsibilities.

1.3 Project Status

So far, all screens have been implemented; however, only the settings screen is the only one that is fully completed. The help screen lacks a detailed and well-structured user manual, which will be written next week. The exiting functionality of the main menu is also functional. Regarding the play option of the main menu, the level menu in it has been established, along with all the level options. The script submitted by the user can be read and run, while no evaluation (star system) of the submitted solutions has been implemented. Lastly, the design and structure of most levels is also something that is still missing from our project.

We believe that so far we have done a good job in following and fulfilling our use case originally declared. The same goes for the requirements, as many of them have already been fulfilled. We are optimistic that we will be done with the project on time, as we already have a functional program that can be built and run. However, there are still details that need to be added for it to be more user-friendly. The designs of most levels are also still under implementation.